

# Traditional Examples

Arrays, 2D/3D, and ArrayLists

---

A menu of neutral, classic examples to run live. Each is self-contained and compiles as written (drop the body into a `main`, or the class as shown). The **Ask the class** and **Predict** lines are optional prompts for the interactive parts: pause, take guesses, *then* run it.

## A. One-dimensional review

### A1. Sum and average of test scores

```
int[] scores = {88, 92, 79, 100, 67, 85};
int total = 0;
for (int s : scores) {
    total += s;
}
double average = (double) total / scores.length;
System.out.printf("Total %d over %d scores, average %.1f%n",
    total, scores.length, average);
```

**Ask the class:** why the `(double)` cast? Delete it and re-run: where did the decimals go, and why? (Integer division happens *before* the assignment to `double`.)

### A2. Find the max (and where it lives)

```
int[] readings = {23, 47, 12, 88, 51, 88, 9};
int maxIndex = 0;
for (int i = 1; i < readings.length; i++) {
    if (readings[i] > readings[maxIndex]) {
        maxIndex = i;
    }
}
System.out.println("Max " + readings[maxIndex] + " first seen at index " + maxIndex);
```

**Ask the class:** there are two 88s. Which index wins, and how would you change the comparison to catch the *last* max instead of the first?

### A3. Reference versus copy: the one that surprises people

```
int[] a = {1, 2, 3};
int[] b = a; // same array
int[] c = java.util.Arrays.copyOf(a, a.length); // independent copy
b[0] = 99;
c[1] = 77;
System.out.println(java.util.Arrays.toString(a)); // Predict before running
```

**Predict:** what does a print? (b shares a's storage; c does not.)

## B. Two-dimensional arrays

### B1. Multiplication table

```
int size = 5;
int[][] table = new int[size][size];
for (int r = 0; r < table.length; r++) {
    for (int c = 0; c < table[r].length; c++) {
        table[r][c] = (r + 1) * (c + 1);
    }
}
System.out.println(java.util.Arrays.deepToString(table));
```

**Ask the class:** `table.length` versus `table[r].length`: what does each measure here, and would you notice the difference if the grid were not square?

### B2. Gradebook: rows are students, columns are assignments

```
String[] students = {"Alice", "Bob", "Chen"};
int[][] grades = {
    {90, 85, 100}, // Alice's three assignments
    {70, 95, 80},  // Bob's
    {88, 88, 88}   // Chen's
};
for (int s = 0; s < students.length; s++) {
    int sum = 0;
    for (int g : grades[s]) {
        sum += g;
    }
    System.out.printf("%-6s average: %.1f%n", students[s], (double) sum / grades[s].length);
}
```

**Ask the class:** how would we instead average each *assignment* down a column? (You cannot use the enhanced for for that: you need the column index, so it is a counted inner loop.)

### B3. Transpose a matrix (flip rows and columns)

```
int[][] m = {
    {1, 2, 3},
    {4, 5, 6}
};
// 2 rows, 3 cols
int[][] t = new int[m[0].length][m.length]; // 3 rows, 2 cols
for (int r = 0; r < m.length; r++) {
    for (int c = 0; c < m[r].length; c++) {
        t[c][r] = m[r][c];
    }
}
System.out.println(java.util.Arrays.deepToString(t)); // [[1, 4], [2, 5], [3, 6]]
```

---

**Ask the class:** why is the new array `[cols][rows]` and not `[rows][cols]`?

## B4. Tic-tac-toe board (2D char grid)

```
char[][] board = {
    {'X', 'O', 'X'},
    {' ', 'X', 'O'},
    {'O', ' ', 'X'}
};
for (char[] row : board) {
    for (char cell : row) {
        System.out.print(cell == ' ' ? ' ' : cell);
        System.out.print(' ');
    }
    System.out.println();
}
```

**Extend live:** write a check for “did X win the main diagonal?”: `board[0][0]`, `board[1][1]`, `board[2][2]`. Great five-minute in-class build.

## C. Ragged arrays

### C1. Pascal’s triangle

```
int rows = 5;
int[][] tri = new int[rows][];           // row lengths not set yet
for (int r = 0; r < rows; r++) {
    tri[r] = new int[r + 1];             // row r has r+1 entries
    tri[r][0] = 1;
    tri[r][r] = 1;
    for (int c = 1; c < r; c++) {
        tri[r][c] = tri[r - 1][c - 1] + tri[r - 1][c];
    }
}
for (int[] row : tri) {
    System.out.println(java.util.Arrays.toString(row));
}
```

**Ask the class:** right after `new int[rows][]`, what is `tri[2]`? (It is null: the outer array of references exists, the rows do not yet.) This is the cleanest motivation for ragged arrays there is.

## D. Three-dimensional arrays

### D1. Temperatures: 2 sensors x 3 days x 4 readings

```
double[][][] temps = new double[2][3][4];
for (int sensor = 0; sensor < temps.length; sensor++) {
    for (int day = 0; day < temps[sensor].length; day++) {
```

```

        for (int reading = 0; reading < temps[sensor][day].length; reading++) {
            temps[sensor][day][reading] = 60 + Math.random() * 40; // 60.0-100.0
        }
    }
}
System.out.printf("Sensor 1, day 2, reading 3 = %.1f%n", temps[1][2][3]);

```

**Ask the class:** name each index out loud: what does `temps[1][2][3]` mean in plain English? Naming the axes is half the battle with 3-D.

## D2. A tiny RGB image (height x width x 3 channels)

```

int h = 2, w = 2;
int[][][] pixels = new int[h][w][3]; // last axis = {red, green, blue}
pixels[0][0] = new int[]{255, 0, 0}; // top-left red
pixels[1][1] = new int[]{0, 0, 255}; // bottom-right blue
System.out.println(java.util.Arrays.deepToString(pixels));

```

**Ask the class:** is the 3-D array here doing real work, or would a 2-D array of a small `Color` class be cleaner? (Good “when do brackets stop paying off” discussion.)

## E. The Arrays utility class

### E1. Sort, then binary-search (order matters)

```

int[] data = {42, 7, 19, 88, 3, 55};
java.util.Arrays.sort(data); // sorts IN PLACE
System.out.println(java.util.Arrays.toString(data));
int pos = java.util.Arrays.binarySearch(data, 55); // valid ONLY because it's sorted
// now
System.out.println("55 is at index " + pos);

```

**Ask the class:** run `binarySearch` *before* sorting on purpose: note it returns something, and note it is wrong. No exception, just a lie. Why is that dangerous?

### E2. equals versus ==

```

int[] p = {1, 2, 3};
int[] q = {1, 2, 3};
System.out.println(p == q); // Predict
System.out.println(java.util.Arrays.equals(p, q)); // Predict

```

**Predict both** before running. (`==` asks “same object?”; `Arrays.equals` asks “same contents?”.)

### E3. “Resizing” with `copyOf`

```

int[] small = {1, 2, 3};
int[] bigger = java.util.Arrays.copyOf(small, 5); // {1, 2, 3, 0, 0}

```

```
System.out.println(java.util.Arrays.toString(bigger));
```

**Tie-in:** this is precisely the move an `ArrayList` makes for you when it runs out of room. Keep it in mind for the next section.

## F. ArrayList

### F1. A to-do list that grows and shrinks

```
import java.util.ArrayList;

ArrayList<String> todo = new ArrayList<>();
todo.add("write lecture");
todo.add("grade projects");
todo.add("coffee");
todo.add(0, "coffee FIRST");           // insert at the front, shifts the rest right
todo.remove("coffee");                 // remove by value
System.out.println(todo);
System.out.println("Items left: " + todo.size());
```

**Ask the class:** predict the printout, then trace the two shifts (the insert and the remove) on the board.

### F2. Reading an unknown number of inputs (the reason `ArrayList` exists)

```
import java.util.ArrayList;
import java.util.Scanner;

Scanner in = new Scanner(System.in);
ArrayList<Integer> nums = new ArrayList<>();
System.out.println("Enter integers, blank line to stop:");
while (in.hasNextInt()) {
    nums.add(in.nextInt());           // don't know the count up front -- arrays can't
    // do this cleanly
}
System.out.println("Read " + nums.size() + " numbers: " + nums);
```

**Ask the class:** how would you do this with a plain array? (You would guess a size and copyOf when you overflow, that is, reimplement `ArrayList` by hand.)

### F3. The `remove(int)` versus `remove(Object)` trap: run it, do not just describe it

```
import java.util.ArrayList;

ArrayList<Integer> nums = new ArrayList<>();
for (int i = 10; i <= 15; i++) nums.add(i); // [10, 11, 12, 13, 14, 15]
nums.remove(2);                             // Predict: what leaves?
nums.remove(Integer.valueOf(13));            // Predict: what leaves?
System.out.println(nums);
```

**Predict each removal.** (`remove(2)` deletes *index* 2 → the value 12. `Integer.valueOf(13)` deletes the *value* 13.) This slide-in-code is the single most valuable 90 seconds in the whole unit.

#### F4. Safe removal while iterating

```
import java.util.ArrayList;
import java.util.Iterator;

ArrayList<String> words = new ArrayList<>(java.util.List.of("a", "", "b", "", "c"));
Iterator<String> it = words.iterator();
while (it.hasNext()) {
    if (it.next().isEmpty()) {
        it.remove();           // safe; the enhanced for + list.remove would throw here
    }
}
System.out.println(words);    // [a, b, c]
// One-liner alternative: words.removeIf(String::isEmpty);
```

**Ask the class:** swap this for a `for (String w : words) words.remove(w);` and run it, then watch the `ConcurrentModificationException`. Seeing the crash is more memorable than the warning.

#### F5. Array versus ArrayList, same task, side by side

```
// Fixed roster of exactly 3 -- array is fine
String[] fixed = {"Mercury", "Venus", "Earth"};

// Roster that will grow as we "discover" more -- ArrayList
import java.util.ArrayList;
ArrayList<String> discovered = new ArrayList<>();
discovered.add("Mercury");
discovered.add("Venus");
discovered.add("Earth");
discovered.add("Mars");           // trivial with a list; a whole copy dance with the
array
System.out.println(fixed.length + " fixed, " + discovered.size() + " discovered");
```

**Ask the class:** which structure would you pick for a chessboard? For a shopping cart? For the pixels of an image? For a guest list at a party? (Grid or fixed → array; grows → list.)